DESARROLLO DE APLICACIONES MÓVILES

FitPulse

Guía del proyecto y preparación para la defensa oral

Una explicación de la arquitectura, los modelos de datos, la lógica de rutinas y calorías, y las preguntas técnicas que conviene dominar.

Proyecto	Aplicación móvil de fitness y nutrición
Tecnologías principales	Expo 54 · React Native 0.81.5 · React 19.1 · TypeScript
Fecha de preparación	7 de octubre de 2026
Base del contenido	Código revisado del repositorio y documentación de Expo SDK 54

La idea central

La aplicación administra rutinas y alimentos mediante estado local de React. El usuario puede crear, editar y eliminar rutinas, y registrar calorías. Todavía no hay almacenamiento permanente, autenticación ni sesiones reales de entrenamiento.

Cómo utilizar esta guía

Estudia primero las secciones 1 a 7. Después practica las 30 preguntas de las secciones 8 a 10 sin leer las respuestas. Explica cada concepto con un ejemplo del código y reconoce las funciones pendientes.

Advertencia documental: el README menciona SDK 57, pero package.json declara Expo ~54.0.37. Esta guía describe la aplicación principal de la raíz, no la plantilla independiente my-expo-app.

Arquitectura y navegación

FitPulse tiene cuatro pestañas. Entrenos y Calorías contienen la lógica funcional principal; Inicio y Perfil muestran datos de ejemplo.

```
expo-router/entry
  src/app/_layout.tsx        Stack global
    src/app/(tabs)/_layout.tsx
      index.tsx              Inicio
      entrenos.tsx           Rutinas
      progreso.tsx           Calorías
      perfil.tsx             Perfil
```

Ubicación	Responsabilidad
src/app/	Pantallas y layouts de navegación.
src/components/	Formulario de rutinas y selector de ejercicios.
src/store/	Interfaces, tipos, catálogo y datos iniciales.
assets/	Imágenes e iconos locales.
my-expo-app/	Proyecto independiente de plantilla; declara Expo SDK 56.

Los dos layouts

Layout raíz: importa global.css, configura un Stack sin encabezado y registra el grupo (tabs). También solicita ocultar la barra de navegación del sistema en Android.

Layout de pestañas: configura Tabs.Screen para Inicio, Entrenos, Calorías y Perfil. Define iconos, colores activos e inactivos, y espaciado por plataforma.

Los paréntesis de (tabs) indican un grupo de rutas y no forman parte del camino de navegación. El archivo progreso.tsx tiene ese nombre, aunque su título visible es Calorías.

Inicio y Perfil

Inicio muestra una meta semanal fija, un entrenamiento recomendado y el texto de prueba "holaaaaa". Perfil muestra un usuario y correo de ejemplo. Sus opciones no tienen onPress, así que no abren vistas ni ejecutan acciones.

La entrada de la aplicación principal es expo-router/entry. El App.tsx de my-expo-app no es su punto de entrada. Esa carpeta está excluida del tsconfig principal.

Modelos de datos y estado

```
Rutina
  id: string
  nombre: string
  ejercicios: Ejercicio[]
    id, nombre, detalle?, imagen?
    series: Serie[]
      id: string
      numeroSerie: number
      peso: number
      repeticiones: number
```

routines.store.ts define las interfaces Serie, Ejercicio y Rutina. rutinasIniciales es un arreglo vacío. exercises.store.ts define EjercicioCatalogo y cinco ejercicios con imágenes locales.

```
type TipoComida = "Desayuno" | "Almuerzo" | "Cena" | "Snack";

interface Alimento {
  id: string;
  nombre: string;
  calorias: number;
  tipoComida: TipoComida;
  hora: string;
}
```

Qué aportan los tipos

Una interface describe la estructura esperada de un objeto. El signo ? indica una propiedad opcional; [] indica un arreglo y | permite alternativas. Los tipos comprueban el código durante el desarrollo, pero no validan automáticamente la entrada del usuario.

La propiedad imagen usa any en el código actual. Esto reduce las comprobaciones de tipos; no es una exigencia general de React Native.

Estado local, no almacenamiento permanente

```
const [rutinas, setRutinas] =
  useState<Rutina[]>(rutinasIniciales);
```

rutinas contiene la lista actual y setRutinas la actualiza. React vuelve a renderizar el componente al actualizar su estado. Los archivos llamados store exportan modelos y valores iniciales; no implementan un estado global ni una base de datos.

Consecuencia: el estado dura mientras se mantiene esa instancia de la aplicación. Al reiniciar, se recuperan los valores iniciales. Inicio no recibe automáticamente un resumen de los cambios de Entrenos.

Entrenos: el flujo CRUD

CRUD significa crear, leer, actualizar y eliminar. entrenos.tsx administra la lista de rutinas, la rutina a editar, la visibilidad del formulario, el menú de opciones y el banner.

Crear o editar

rutinaAEditar vale null al crear y contiene una Rutina al editar. abrirModalCrear limpia la selección; abrirModalEditar precarga la rutina elegida y cierra el menú de opciones.

```
if (rutinaAEditar) {
  setRutinas(
    rutinas.map((r) =>
      r.id === rutinaGuardada.id ? rutinaGuardada : r
    )
  );
} else {
  setRutinas([rutinaGuardada, ...rutinas]);
}
```

Al editar, map conserva cada rutina salvo la que tiene el mismo ID, que se reemplaza por la guardada. Al crear, el operador spread conserva los elementos anteriores y agrega la nueva al principio.

Eliminar

```
setRutinas(rutinas.filter((r) => r.id !== rutina.id));
```

Después de una confirmación, filter construye un arreglo que excluye la rutina seleccionada. El ID se utiliza para identificar el objeto, no su nombre, que podría repetirse.

Visualización y funciones pendientes

Cada tarjeta utiliza key={rutina.id}, muestra hasta tres ejercicios con slice(0, 3) e indica si hay más. Si no existen rutinas, aparece un estado vacío con una acción para crear la primera.

Empezar Rutina y Empezar Entrenamiento Vacío solo muestran un Alert. Explorar también es una simulación. El botón de refrescar no tiene acción y el banner anuncia reordenamiento sin una implementación asociada.

Inmutabilidad

map, filter y spread crean estructuras nuevas. Evitar modificar directamente el estado facilita que React identifique cambios y reduce alteraciones accidentales de datos compartidos. Cuando el nuevo estado depende del anterior, una actualización funcional con el valor previo es una alternativa robusta.

El formulario de rutinas

RoutineModal.tsx contiene ModalRutina, el componente con la lógica más compleja. Recibe props desde Entrenos y mantiene un borrador local del nombre y los ejercicios.

```
interface PropiedadesModalRutina {
  visible: boolean;
  rutinaAEditar: Rutina | null;
  alGuardar: (rutina: Rutina) => void;
  alCerrar: () => void;
}
```

Al abrirse, onShow prepara el formulario. En edición, copia los ejercicios con JSON.parse(JSON.stringify(...)); en creación, precarga el primer ejercicio del catálogo con tres series de 20 kg y 10 repeticiones.

La copia separa el borrador del original. JSON sirve para los datos simples actuales, pero no conserva todos los tipos de JavaScript. Una copia explícita de ejercicios y series haría más clara esta estructura.

Función	Comportamiento
alSeleccionarEjercicio	Añade un ejercicio con tres series iniciales.
agregarSerie	Copia peso y repeticiones de la última serie.
eliminarSerie	Evita quitar la última y renumera las restantes.
eliminarEjercicio	Evita quitar el último ejercicio.
actualizarPesoSerie / repeticiones	Actualiza una serie del ejercicio seleccionado.
guardarRutina	Valida nombre y ejercicios; conserva o genera el ID.

La actualización anidada más importante

```
return {
  ...ej,
  series: ej.series.map((s) =>
    s.id === idSerie ? { ...s, peso: valorPeso } : s
  ),
};
```

Dentro del map de ejercicios, esta expresión crea un ejercicio nuevo, un arreglo nuevo de series y una serie nueva con el peso actualizado. Las demás propiedades se conservan.

Al guardar, alGuardar(rutinaGuardada) devuelve el resultado a Entrenos. Al cancelar, el borrador no se incorpora a la lista principal. Los valores numéricos se convierten con parseFloat y parseInt; aún faltan reglas estrictas para pesos y repeticiones negativos.

Catálogo y contador de calorías

Selector de ejercicios

ExercisePickerModal.tsx filtra el catálogo por nombre o descripción. toLowerCase ignora diferencias de mayúsculas y includes busca coincidencias parciales. No se normalizan las tildes.

```
catalogoEjercicios.filter((ej) =>
  ej.nombre.toLowerCase().includes(busqueda.toLowerCase()) ||
  ej.detalle.toLowerCase().includes(busqueda.toLowerCase())
);
```

FlatList presenta los resultados con ID, imagen, nombre y detalle. Al seleccionar, llama a alSeleccionar(item), cierra y limpia la búsqueda. Las imágenes locales vienen de require y se muestran con expo-image.

Calorías y meta diaria

progreso.tsx administra alimentos, meta diaria, filtro de categoría y dos formularios: registro de alimento y cambio de meta. La meta inicial es 2.000 kcal.

```
const totalConsumido = alimentos.reduce(
  (total, item) => total + item.calorias, 0
);
const caloriasRestantes =
  Math.max(0, metaDiaria - totalConsumido);
const porcentajeProgreso = Math.min(
  100, Math.round((totalConsumido / metaDiaria) * 100)
);
```

Los alimentos iniciales suman 1.120 kcal: con la meta de 2.000, quedan 880 y el progreso es 56 %. Si se supera la meta, la barra se vuelve roja; las restantes no bajan de cero y el porcentaje no supera 100.

Filtrar por categoría cambia solo la lista visible. El resumen sigue sumando todos los alimentos. Los cálculos son valores derivados y se recalculan al renderizar; no requieren un estado independiente.

Validación y alcance

Un alimento requiere nombre y calorías mayores que cero. La meta admite al menos 500 según la regla implementada. Registrar y eliminar actualizan el estado; Limpiar día elimina la lista tras confirmar.

Cada alimento almacena una hora HH:MM, pero no una fecha completa. Por tanto, no existe separación automática por día ni historial. Las calorías se ingresan manualmente: no se consultan desde una base nutricional.

Interfaz y configuración técnica

Los componentes que debes reconocer

Elemento	Función
View / Text	Contenedor de interfaz y texto móvil. JSX no equivale a HTML.
TextInput	Campo controlado mediante value y onChangeText.
TouchableOpacity / Pressable	Acciones táctiles; necesitan handlers para ejecutar lógica.
Modal / Alert	Formulario superpuesto y diálogo de aviso o confirmación.
ScrollView / FlatList	Contenido desplazable y lista con renderizado por ventana.
SafeAreaView	Respeta zonas seguras del dispositivo.
KeyboardAvoidingView	Ajusta el contenido frente al teclado según plataforma.

```
<TextInput
  value={nombre}
  onChangeText={setNombre}
/>

{rutinas.length === 0 && <VistaVacia />}
```

El campo controlado refleja el estado y lo actualiza al escribir. La expresión con && muestra una vista solo si la condición es verdadera. key identifica elementos de una lista; ?. accede a una propiedad si el objeto existe.

Cómo se conectan los estilos

NativeWind permite className="bg-orange-500 px-4 py-3 rounded-xl". global.css incluye las directivas Tailwind; tailwind.config.js examina src; Babel transforma el código y Metro conecta NativeWind con el CSS. nativewind-env.d.ts añade soporte de tipos.

Los archivos de configuración

app.json: nombre, iconos, orientación, splash, esquema de enlaces y opciones por plataforma. **tsconfig.json:** comprobación estricta y alias. **eslint.config.js:** reglas de revisión. **package-lock.json:** versiones resueltas de dependencias.

El alias @/* apunta a src y @/assets/* apunta a assets de la raíz. node_modules contiene dependencias; .expo contiene datos generados. AGENTS.md establece instrucciones de trabajo; README explica el proyecto y LICENSE define sus condiciones de uso.

Limitaciones, verificación y estudio

Qué funciona y qué queda pendiente

Implementado	Pendiente
CRUD de rutinas y series	Persistencia y recuperación de datos.
Catálogo y búsqueda de ejercicios	Sesiones de entrenamiento e historial.
Registro, filtros y total de calorías	Separación real por fecha y perfil de usuario.
Formulario de meta y estados vacíos	Reordenamiento, explorador y acciones de Perfil.

Las principales mejoras son conservar los datos, reforzar la validación numérica, conectar Inicio con datos reales y completar el seguimiento de entrenamientos. Los ID basados en Date.now son simples y no garantizan unicidad absoluta ante creaciones simultáneas.

Qué verificaciones se hicieron

En la revisión del proyecto se ejecutaron expo lint y tsc --noEmit sin errores. Son comprobaciones estáticas de la aplicación principal; no constituyen una prueba de todas las interacciones en un teléfono.

```
npx expo start           # Iniciar desarrollo
npx expo lint            # Revisar el código
npx tsc --noEmit          # Comprobar tipos
npx expo install paquete # Instalar versión compatible
```

La raíz contiene package-lock.json y no bun.lock, por lo que se muestran comandos con npx. El script reset-project apunta a scripts/reset-project.js, que no aparece entre los archivos revisados; conviene comprobarlo antes de usarlo.

Orden recomendado de estudio

1. Modelos de rutinas. 2. Estado y CRUD en Entrenos. 3. Props y cambios anidados en RoutineModal. 4. Búsqueda del catálogo. 5. Cálculos de calorías. 6. Layouts de navegación. 7. Configuración y limitaciones.

Referencias

Fuentes locales: package.json; src/app y sus layouts; src/components; src/store; app.json y configuraciones de la raíz. La guía refleja el código revisado, no funciones futuras.

Expo SDK 54

Expo Router para SDK 54

NavigationBar para SDK 54

Defensa oral: fundamentos

Respuestas modelo. Adáptalas a tus palabras y acompáñalas con un ejemplo concreto del proyecto.

1. ¿Cuál es el objetivo de FitPulse?

Organizar rutinas con ejercicios y series, y registrar alimentos para comparar las calorías consumidas con una meta. Los datos actuales se mantienen en memoria.

2. ¿Qué aporta cada tecnología?

React administra componentes y estado; React Native ofrece elementos móviles; Expo facilita el desarrollo; Router organiza rutas; TypeScript comprueba tipos y NativeWind aplica estilos.

3. ¿Qué versión de Expo utiliza?

La aplicación principal declara ~54.0.37. El README menciona SDK 57 y está desactualizado; my-expo-app es independiente y declara SDK 56.

4. ¿Cómo se organiza el código?

app contiene pantallas y navegación, components contiene piezas de interfaz, store contiene modelos y listas iniciales, y assets contiene recursos locales.

5. ¿Cómo funciona la navegación?

expo-router/entry carga el layout raíz con Stack. Dentro, el layout de (tabs) define las cuatro pestañas. El nombre del grupo no forma parte del camino de navegación.

6. ¿Qué diferencia hay entre pantalla y componente?

Una pantalla es una sección navegable como Entrenos. Un componente es una pieza utilizada por una pantalla, como ModalRutina, con su propia responsabilidad.

7. ¿Qué es el estado?

Son datos que cambian mientras se usa un componente. useState entrega el valor y un setter. Actualizarlo provoca un nuevo renderizado para reflejar el cambio.

8. ¿Qué diferencia hay entre estado y props?

El estado lo administra el componente; las props llegan desde su padre. Entrenos pasa al modal la visibilidad, la rutina elegida y callbacks para guardar o cerrar.

9. ¿Cómo se comunica el modal con Entrenos?

Mediante alGuardar(rutinaGuardada). El modal construye el resultado y el padre actualiza la colección. alCerrar comunica que el formulario debe cerrarse.

10. ¿Cómo se modela una rutina?

Una Rutina tiene ejercicios y cada Ejercicio tiene series. Una Serie registra ID, número de serie, peso y repeticiones. Esto permite cargas distintas por ejercicio y serie.

Defensa oral: lógica y datos

Respuestas modelo. Adáptalas a tus palabras y acompáñalas con un ejemplo concreto del proyecto.

11. ¿Para qué sirven las interfaces?

Describen la estructura de datos y permiten detectar incompatibilidades durante el desarrollo. No validan por sí solas el texto o números ingresados por el usuario.

12. ¿Qué significa CRUD?

Crear, leer, actualizar y eliminar. Entrenos crea rutinas, las muestra, reemplaza sus datos al editar y las quita al eliminar, siempre en el estado local.

13. ¿Cómo distingues creación y edición?

rutinaAEditar es null al crear. Al editar contiene una Rutina, que se precarga en el formulario y conserva su ID al guardar.

14. ¿Para qué usas map, filter y spread?

map transforma o reemplaza elementos; filter conserva los que cumplen una condición; spread copia elementos o propiedades. Se usan para construir el siguiente estado.

15. ¿Qué es la inmutabilidad?

Actualizar creando arreglos y objetos nuevos en vez de alterar directamente los existentes. Para una serie, se reconstruye el ejercicio, su lista de series y la serie modificada.

16. ¿Qué evita que cancelar cambie la rutina?

El modal mantiene un borrador y copia los ejercicios al editar. Solo comunica el resultado al guardar. La copia por JSON es válida para estos datos simples, pero tiene limitaciones.

17. ¿Cómo funciona la búsqueda?

Filtra el catálogo por coincidencias en nombre o descripción mediante includes y toLowerCase. Ignora mayúsculas, pero no normaliza las tildes.

18. ¿Por qué utilizar FlatList?

Mantiene una ventana de elementos renderizados, útil al crecer el catálogo. ScrollView con map genera todos los elementos; la elección depende del tamaño y de la implementación.

19. ¿Cómo se calculan las calorías?

reduce suma las kcal; la meta menos el total da las restantes; total dividido por meta y multiplicado por cien da el porcentaje. Se limitan restantes a cero y porcentaje a cien.

20. ¿El filtro cambia el total?

No. Cambia la lista visible, pero el cálculo del resumen utiliza todos los alimentos. Es una separación entre los datos originales y su presentación filtrada.

Defensa oral: límites y mejoras

Respuestas modelo. Adáptalas a tus palabras y acompáñalas con un ejemplo concreto del proyecto.

21. ¿Qué validaciones implementaste?

Nombre y al menos un ejercicio por rutina; al menos una serie por ejercicio; nombre y calorías positivas por alimento. Falta reforzar la validación de pesos y repeticiones.

22. ¿El teclado numérico garantiza números válidos?

No. Facilita la entrada, pero la lógica debe validar el valor y sus límites. parseInt puede aceptar parcialmente texto como 100abc y no sustituye una validación estricta.

23. ¿Cómo consideras Android e iOS?

Platform.OS ajusta el comportamiento del teclado y el espaciado de pestañas. SafeAreaView protege el contenido de zonas del dispositivo. Debe comprobarse en ambas plataformas.

24. ¿Qué hace useEffect en el layout?

Solicita ocultar la barra de navegación del sistema en Android después de montar el componente. El efecto tiene dependencias vacías y no depende del estado cambiante de la pantalla.

25. ¿Se conservan los datos al reiniciar?

No. useState es temporal. Sería necesario implementar almacenamiento persistente y recuperar las rutinas, alimentos y preferencias al iniciar la aplicación.

26. ¿La carpeta store es una base de datos?

No. Define interfaces, catálogo y datos iniciales. No implementa Redux, Zustand ni un estado global compartido; cada pantalla administra su estado local.

27. ¿Qué funciones están incompletas?

Inicio y Perfil usan datos de ejemplo. Empezar entrenamiento y Explorar muestran avisos; no hay sesión real, reordenamiento ni acciones de las opciones del perfil.

28. ¿El registro distingue cada día?

No automáticamente. Guarda hora, pero no fecha completa. Para un historial habría que registrar fechas y agrupar o filtrar los alimentos por día.

29. ¿Qué mejorarías y cómo lo probarías?

Priorizaría persistencia, validación y sesiones reales. Probaría crear, editar, cancelar y eliminar rutinas, gestionar series y registrar, filtrar y eliminar alimentos en dispositivos.

30. ¿Cuál es el código más importante?

RoutineModal, porque combina props, estado del borrador, callbacks y actualizaciones inmutables anidadas. También son centrales el CRUD de Entrenos y los cálculos derivados de Calorías.